

Competitive Programming

Section 17: Topological Sorting And DAG DP

Competitive Programming Course

2026-08-05

Table of contents

1	Why topological order matters	3
2	Recognition	4
3	Vocabulary	5
3.1	DAG	5
3.2	In-degree	5
4	Kahn's algorithm	6
5	Queue or heap?	7
6	Worked example 1: topological order	8
7	Worked example 2: cycle detection	9
8	DAG dynamic programming	10
9	Earliest finish times	11
10	DFS topological order	12
11	Complexity	13
12	Implementation notes	14
12.1	C++	14
12.2	Python	14
13	Local exercises	15
13.1	A. Lexicographic Course Order	15
13.2	B. Longest DAG Path	15
13.3	C. Project Schedule	15
14	Submission checklist	16
15	Exercises	17

1 Why topological order matters

Many contest problems are dependency problems:

```
course A before course B  
job X before job Y  
build library before program  
state u must be solved before state v
```

If dependencies form a directed acyclic graph, or DAG, we can order the vertices so every edge points forward. That order is called a topological order.

Once we have such an order, dynamic programming becomes straightforward:

```
process prerequisites before dependents  
push answers along outgoing edges
```

2 Recognition

Look for these signals:

Statement signal	Likely tool
prerequisites, dependencies, must happen before	topological sorting
impossible if circular dependency exists	cycle detection via topo count
longest path in a DAG	DP in topological order
earliest finish time with job durations	topological DP
directed graph with no cycles guaranteed	topological order, often DP

Warning:

Longest path is hard in a general directed graph.
Longest path in a DAG is easy because topological order removes cycles.

3 Vocabulary

3.1 DAG

A DAG is a directed acyclic graph:

```
Directed: edges have direction u -> v
Acyclic: no directed cycle exists
```

Example DAG:

```
1 -> 3 -> 5
2 -> 3
2 -> 4 -> 5
```

Example with a cycle:

```
1 -> 2 -> 3 -> 1
```

No topological order exists for the second graph because each vertex in the cycle must come before another vertex in the same cycle.

3.2 In-degree

The in-degree of a vertex is the number of incoming edges.

For:

```
1 -> 3
2 -> 3
2 -> 4
```

the in-degrees are:

Vertex	In-degree
1	0
2	0
3	2
4	1

A vertex with in-degree 0 currently has no unmet prerequisites.

4 Kahn's algorithm

Kahn's algorithm builds a topological order by repeatedly taking a vertex with in-degree 0.

Template:

```
compute indegree[v] for all vertices
put all vertices with indegree 0 into a queue
order = []

while queue not empty:
    u = pop one available vertex
    append u to order
    for each edge u -> v:
        indegree[v] -= 1
        if indegree[v] == 0:
            push v

if order has all vertices:
    order is topological
else:
    a cycle exists
```

Why the cycle test works:

If a cycle remains, every vertex in that cycle still has an incoming edge from inside the cycle, so none of them can become indegree 0.

5 Queue or heap?

If the problem asks for any valid topological order, use a queue.

If the problem asks for the lexicographically smallest order, use a min-heap.

A **heap** is a data structure that stores a collection of values and lets us quickly access and remove the value with highest priority. In a **min-heap**, the smallest value has highest priority:

<code>push(x)</code>	<code>insert x</code>	<code>$O(\log n)$</code>
<code>top</code>	<code>inspect the smallest value</code>	<code>$O(1)$</code>
<code>pop</code>	<code>remove the smallest value</code>	<code>$O(\log n)$</code>

Unlike a sorted array, a heap does not keep every value in sorted order. It only guarantees that the smallest value can be accessed immediately. That is exactly what Kahn's algorithm needs: among all currently available vertices, repeatedly take the smallest one.

The min-heap is not a new graph idea. It is just a way to choose the smallest currently available course.

6 Worked example 1: topological order

Dependencies:

```
1 -> 3
2 -> 3
2 -> 4
4 -> 5
```

Initial in-degrees:

```
1:0  2:0  3:2  4:1  5:1
```

Using a min-heap:

```
available = [1,2]
take 1 -> order [1], indeg[3] becomes 1
available = [2]
take 2 -> order [1,2], indeg[3] becomes 0, indeg[4] becomes 0
available = [3,4]
take 3 -> order [1,2,3]
take 4 -> order [1,2,3,4], indeg[5] becomes 0
take 5 -> order [1,2,3,4,5]
```

Every edge points forward in the final order.

7 Worked example 2: cycle detection

Graph:

```
1 -> 2 -> 3 -> 1
4 -> 5
```

Initial in-degrees:

```
1:1  2:1  3:1  4:0  5:1
```

Kahn's algorithm processes 4, then 5. After that:

```
order size = 2, n = 5
```

Vertices 1,2,3 never become available. That proves a cycle exists.

8 DAG dynamic programming

Topological order turns a graph dependency problem into a one-pass DP.

For longest path length in edges:

```
dp[u] = length of longest path ending at u
```

When processing edge $u \rightarrow v$:

```
dp[v] = max(dp[v], dp[u] + 1)
```

Because u is processed before v , all ways to reach u are already known.

Diagram:

```
1 -> 2 -> 4 -> 5
|           ^
v           |
3 -----+
```

Topological order can be:

```
1,2,3,4,5
```

DP updates:

```
dp[1]=0
1->2: dp[2]=1
1->3: dp[3]=1
2->4: dp[4]=2
3->4: dp[4]=2
4->5: dp[5]=3
```

Answer: 3.

9 Earliest finish times

For scheduling with durations:

```
finish[u] = earliest time job u can finish  
start[u]  = earliest time job u can start
```

If $u \rightarrow v$, then v cannot start until u finishes:

```
start[v] = max(start[v], finish[u])  
finish[u] = start[u] + duration[u]
```

This is DAG DP with `max` over prerequisites.

10 DFS topological order

There is also a DFS-based topological sort:

```
run DFS
append vertex when it finishes
reverse finish order
```

This connects to Section 16's postorder idea. This section's reference solutions use Kahn's algorithm because it detects cycles naturally and supports lexicographically smallest order with a heap.

11 Complexity

With adjacency lists:

```
Time with queue:  $O(n + m)$   
Time with min-heap:  $O((n + m) \log n)$   
Memory:  $O(n + m)$ 
```

The heap version is slower but needed when the smallest available vertex must be chosen at every step.

12 Implementation notes

12.1 C++

Queue version:

```
queue<int> q;
```

Min-heap version:

```
priority_queue<int, vector<int>, greater<int>> pq;
```

For scheduling times, use `long long`:

```
vector<long long> duration(n), finish(n);
```

12.2 Python

Queue version:

```
from collections import deque
q = deque(...)
```

Min-heap:

```
import heapq

heap = []
heapq.heappush(heap, x)
x = heapq.heappop(heap)
```

Python integers already grow as needed, but the algorithm should still be linear or near-linear.

13 Local exercises

13.1 A. Lexicographic Course Order

Use Kahn's algorithm with a min-heap. If all courses are processed, print the order. Otherwise print IMPOSSIBLE.

13.2 B. Longest DAG Path

The graph is guaranteed acyclic. Use Kahn's algorithm to process vertices and update:

```
dp[v] = max(dp[v], dp[u] + 1)
```

13.3 C. Project Schedule

Use Kahn's algorithm. If a cycle exists, print IMPOSSIBLE. Otherwise compute earliest finish times by pushing maximum predecessor finish times forward.

14 Submission checklist

Before submitting:

```
Did I store directed edges in the correct direction?  
Did I increment indegree of the target, not the source?  
Did I initialize all indegree-0 vertices?  
Did I check order size to detect cycles when cycles are possible?  
For lexicographic order, did I use a min-heap instead of a queue?  
For scheduling, did I use 64-bit integers in C++?
```

15 Exercises

Solve A with Kahn's algorithm and a min-heap tie rule, B by propagating DP in topological order, and C by checking whether every dependency is available before scheduling. Test an isolated vertex and a directed cycle.